TataStrive Analytics

Complete product, user, and technical manual

Version 3.0.0

TataStrive

April 9, 2026

Abstract

This manual describes TataStrive Analytics (3.0.0): a Windows-focused desktop application for classroom engagement analysis and cross-day attendance tracking from video folders. It explains what the product does, what operators receive as outputs, how to use each major screen (with screenshots from the project `Screenshots/` folder), end-to-end workflows, the technology stack, and detailed setup, configuration, BigQuery sync, updating, and troubleshooting for developers and IT.

Contents

1 Product overview **3**
- 1.1 What is TataStrive Analytics? . . . 3
- 1.2 Primary use cases . . . 3
- 1.3 Who should use it . . . 3
- 1.4 What you get (deliverables) . . . 3
- 1.5 First launch . . . 3

2 How to use: flows and screenshots **5**
- 2.1 High-level user journey . . . 5
- 2.2 Classroom Analysis tab . . . 5
  - 2.2.1 Screenshot walkthrough . . . 5
  - 2.2.2 Step-by-step . . . 5
  - 2.2.3 Parameters . . . 5
- 2.3 Attendance only tab . . . 5
  - 2.3.1 Screenshot walkthrough . . . 5
  - 2.3.2 Step-by-step . . . 8
- 2.4 Settings dialog . . . 8
- 2.5 Report Viewer . . . 8
- 2.6 BigQuery and menus . . . 10
- 2.7 Keyboard shortcuts . . . 10
- 2.8 Operational playbooks . . . 10
  - 2.8.1 New center onboarding . . . 10
  - 2.8.2 Weak PCs and large files . . . 11

3 How it is made: technology stack **12**
- 3.1 Platform and language . . . 12
- 3.2 Computer vision and ML . . . 12
- 3.3 Optional and cloud . . . 12
- 3.4 Repository layout (maintainers) . . . 12
- 3.5 Architecture diagram . . . 13
- 3.6 Processing workers (conceptual) . . . 13
- 3.7 Frozen executable notes . . . 13

4 Technical documentation **15**
- 4.1 Installation from source . . . 15
  - 4.1.1 Prerequisites . . . 15
  - 4.1.2 Create environment and install dependencies . . . 15
  - 4.1.3 Model files (`Models/`) . . . 15
  - 4.1.4 Launch (development) . . . 15
- 4.2 Building a standalone executable . . . 16
- 4.3 Windows installer (Inno Setup) . . . 16

4.4 Repository and folder structure . . . 16
4.4.1 Top-level layout . . . 16
4.4.2 Application package (`app/`) . . . 16
4.4.3 Classroom analysis helpers (`classroom_analysis/`) . . . 17
4.4.4 Maintenance and ops scripts (`scripts/`) . . . 18
4.4.5 Installer and database SQL (`installer/`) . . . 18
4.4.6 Models directory (`Models/`) . . . 18
4.4.7 Build outputs . . . 18
4.4.8 Per-user configuration (outside the repo) . . . 19
4.4.9 Operator-chosen directories (data plane) . . . 19
4.4.10 Typical output artifacts (by mode) . . . 19
4.4.11 Documentation assets . . . 19
4.4.12 CI . . . 19
4.5 Configuration file . . . 20
4.5.1 Location . . . 20
4.5.2 Top-level keys (summary) . . . 20
4.6 BigQuery integration . . . 20
4.6.1 Module and dataset . . . 20
4.6.2 Tables . . . 20
4.6.3 When sync runs . . . 20
4.7 Updates and releases . . . 21
4.8 Security and privacy . . . 21
4.9 Troubleshooting . . . 21
4.10 How to update the application or dependencies . . . 21
4.10.1 Developer workstation . . . 21
4.10.2 Frozen deployment . . . 22
4.10.3 Configuration migration . . . 22
4.11 Related documentation in the repository . . . 22

A Version and license **23**

Chapter 1

Product overview

1.1 What is TataStrive Analytics?

TataStrive Analytics is a Windows desktop application (version 3.0.0) for **classroom engagement analysis** and **multi-day attendance tracking** from CCTV or training-center recordings. It turns video folders into structured JSON reports, optional annotated video, and—when configured—rows in Google BigQuery for centralized analytics across many physical centers.

1.2 Primary use cases

- **Classroom dynamics:** Periodically "probes" video to estimate presence, activity modes (e.g. lecture vs. activity), and engagement-style signals; writes dashboard-ready JSON.
- **Cross-day attendance:** Detects and tracks faces over time, maintains a **master identity database**, labels returning vs. visitors, optionally matches a known student embedding gallery, and emits daily attendance JSON.
- **Operational monitoring: Folder listening** picks up new files without manual per-file clicks; completed videos are remembered so restarts and updates do not reprocess the same file.
- **Reporting:** Built-in **Report Viewer** opens JSON in table/tree form and supports CSV export.
- **Enterprise analytics:** Optional **BigQuery sync** uploads attendance and engagement summaries; every row is tagged with a configurable Center ID.
- **Maintenance:** Optional **silent updates** from GitHub Releases apply delta patches without a full reinstall.

1.3 Who should use it

1.4 What you get (deliverables)

1.5 First launch

On first run the application prompts for a Center ID; this value is stored in the user profile under `.tatastrive/config.json` on Windows and is attached to BigQuery rows. PyTorch must load for analysis tabs; if initialization fails, the app can offer **limited mode** (Report Viewer and configuration without full ML).

Role	Typical workflow
Center operator	Configure Center ID; set watch and output folders; start monitoring; use BigQuery sync when cloud reporting is enabled.
Analyst / manager	Use Report Viewer locally or query BigQuery for trends across centers and dates.
IT / deployment	Build executable and installer; tune `config.json` and deployment layout per organizational policy.

Table 1.1: Stakeholder roles

Artifact	Description
`class_dynamics_report.json`	Main classroom report (probes, modes, timings).
`stitching_index.json`	Track stitching / raw association data.
`*_attendance_report.json`	Daily attendance summary per run.
Master DB (`.db` / `.pkl`)	Persistent face gallery for cross-day identity.
Optional MP4	Annotated output video (attendance path).
BigQuery tables	`attendance_reports`, `engagement_reports`, `sync_log` (when enabled).

Table 1.2: Primary outputs

Chapter 2

How to use: flows and screenshots

2.1 High-level user journey

Figure 2.1 summarizes the operator path from install to sync and updates.

2.2 Classroom Analysis tab

2.2.1 Screenshot walkthrough

Figure 2.2 shows the **Classroom Analysis** tab.

2.2.2 Step-by-step

1. Open the **Classroom Analysis** tab (`Ctrl+1`).
2. Click **Browse** for **Video Input Folder** and select the directory where new recordings appear.
3. Choose an **Output Directory** for JSON (and related artifacts).
4. Optional: enable **Enable Preview** for a live frame preview (higher CPU/GPU use).
5. Click **Start Monitoring**. The app polls the folder (interval from settings) and processes new videos, skipping files already listed as completed for that folder.
6. Watch **Progress** and the **log** panel; use **Clear** if the log is noisy.
7. When satisfied, use **Report Viewer** to open outputs or **Sync to BigQuery** from the toolbar or BigQuery menu.

2.2.3 Parameters

Defaults and tuning for probe duration, probe interval, frame skip, stitching thresholds, and optional "delete source after process" are in **Settings** → Classroom (and related inference options). See Chapter 4.

2.3 Attendance only tab

2.3.1 Screenshot walkthrough

Figure 2.4 shows the **Attendance only** tab during folder listening.

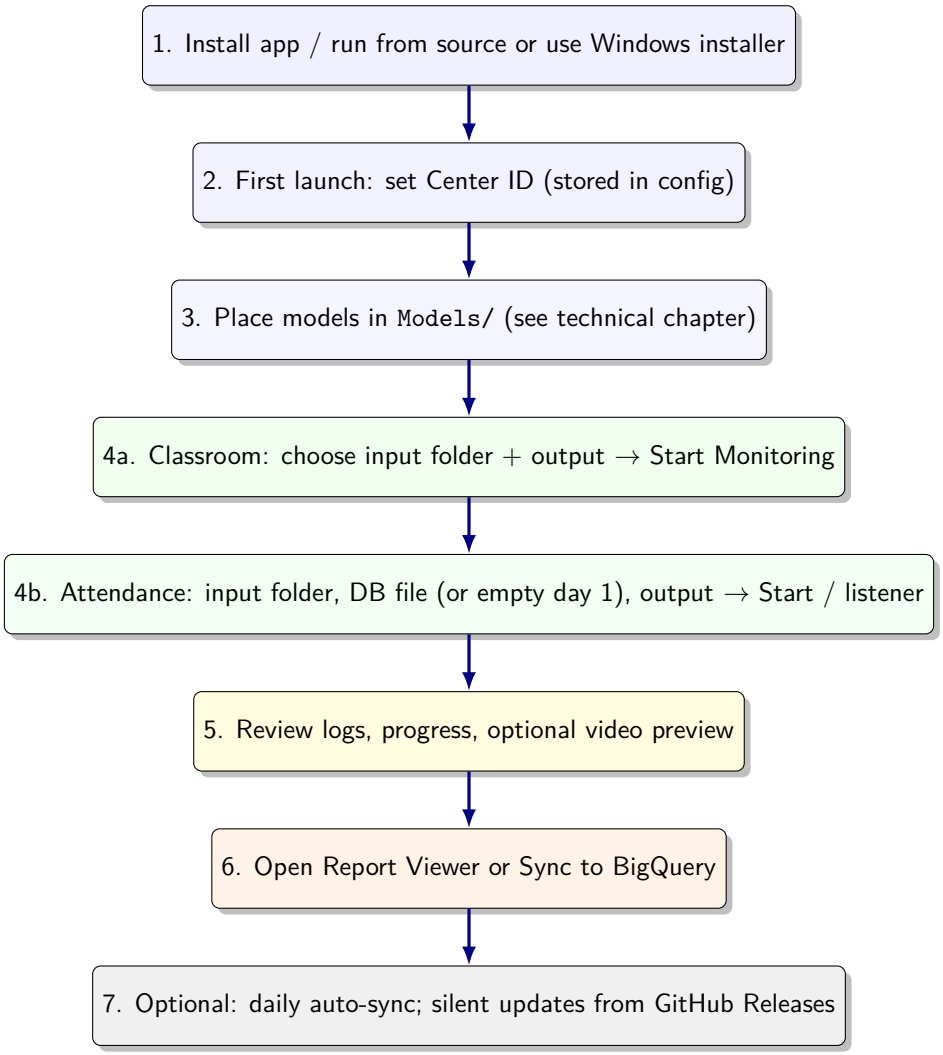

Figure 2.1: End-to-end operator journey from installation through monitoring, reporting, and cloud sync.

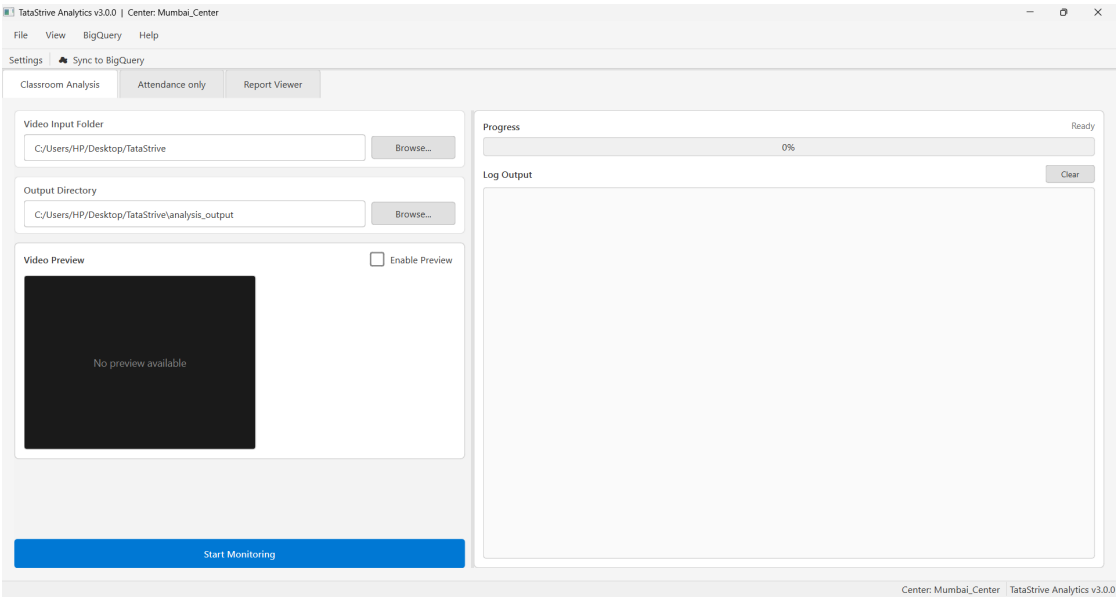

Figure 2.2: **Classroom Analysis:** title bar shows app version and Center ID (`Mumbai_Center` in this example). Use **Video Input Folder** and **Output Directory** (Browse buttons). Optionally enable **Video Preview**. Press **Start Monitoring** to begin folder listening. Progress and logs appear on the right; use **Clear** to reset the log. Toolbar: **Settings**, **Sync to BigQuery**. Menus: File, View, BigQuery, Help.

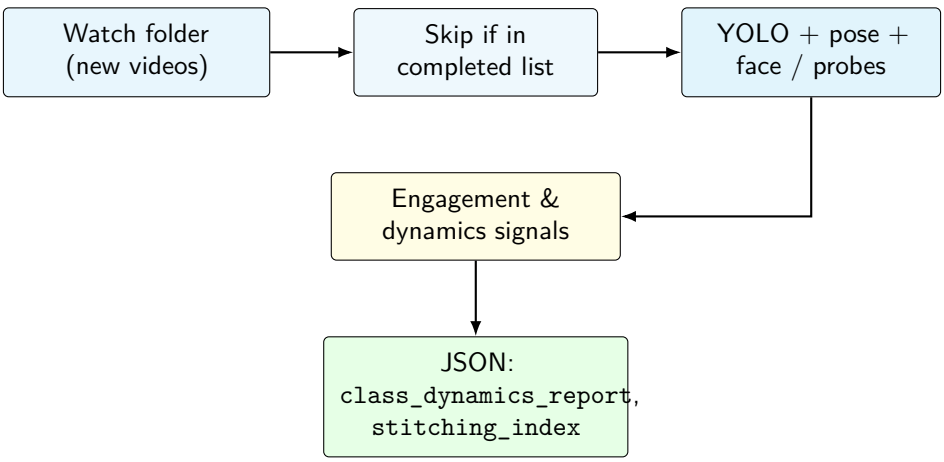

Figure 2.3: Classroom analysis data flow: folder listener, idempotent skip list, vision inference, structured JSON outputs.

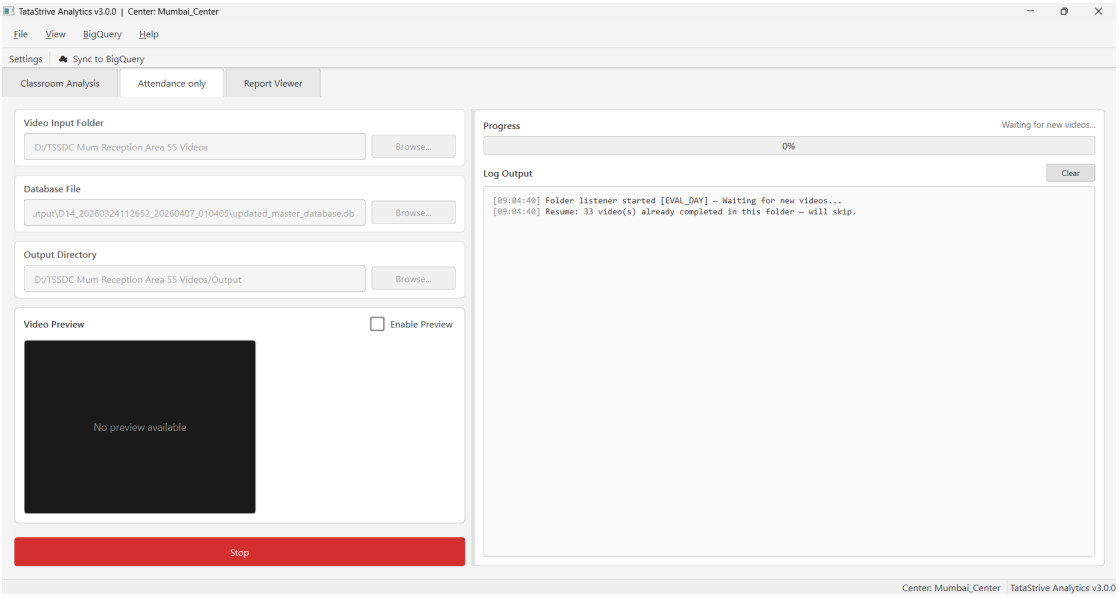

Figure 2.4: **Attendance only:** set **Video Input Folder**, **Database File** (master gallery for evaluation days, or empty/first-day flow per product logic), and **Output Directory**. **Enable Preview** is optional. When listening, status shows **Waiting for new videos...**; the log records `Folder listener started [EVAL_DAY]` and may resume skipping already completed files. The primary action button shows **Stop** while listening or processing.

2.3.2 Step-by-step

1. Open **Attendance only** (`Ctrl+2`).
2. Set **Video Input Folder** to the ingest directory.
3. **Database file:** For **day one** / building the gallery, leave empty or follow your deployment playbook; for **ongoing days**, point to the same logical `.db` or `.pkl` master database produced earlier.
4. Set **Output Directory** for JSON, optional video, crops, and verification folders.
5. Start the run / listener (button label depends on state). The app auto-selects **BUILD_DB** vs. **EVAL_DAY** based on whether a database path is present.
6. Monitor logs; completed videos are tracked per folder to avoid duplicate work.
7. Use **Report Viewer** or BigQuery sync for downstream use.

2.4 Settings dialog

Figure 2.6 shows the **Settings** window (Classroom sub-tab). Other sub-tabs cover Attendance thresholds, Performance (inference device, YOLO/face sizes, frame skip, OpenVINO, preview backend), and General (window geometry, BigQuery auto-sync schedule, etc.).

2.5 Report Viewer

- Open the **Report Viewer** tab (`Ctrl+3`).

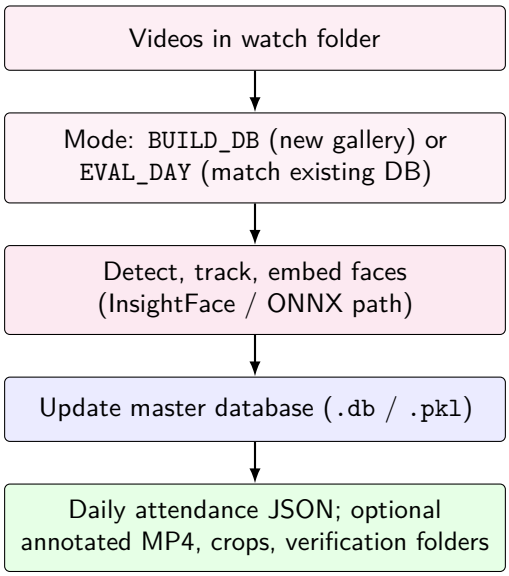

Figure 2.5: Cross-day attendance pipeline: mode depends on whether a master database path is configured; outputs include daily reports and an updated gallery.

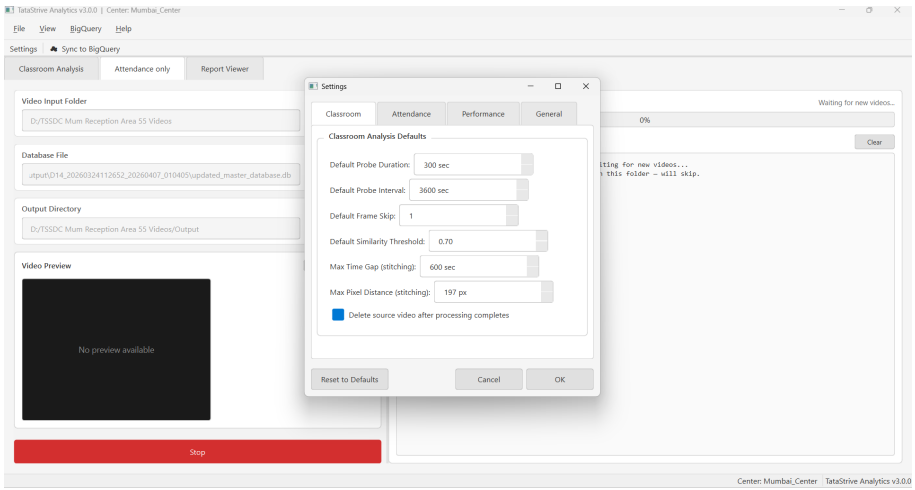

Figure 2.6: **Settings** (Classroom tab): **Default Probe Duration**, **Default Probe Interval**, **Default Frame Skip**, **Default Similarity Threshold**, **Max Time Gap** and **Max Pixel Distance** for stitching, and **Delete source video after processing completes**. Buttons: **Reset to Defaults**, **Cancel**, **OK**. Open via toolbar or `Ctrl+,`.

- Load JSON via File menu / shortcuts or let the app scan last known output directories for patterns such as `*attendance_report*.json`, `*class_dynamics_report*.json`, `*management_summary_report*.json`.
- Switch between table and tree views where applicable; export to CSV when needed.

2.6 BigQuery and menus

- **Sync to BigQuery:** Toolbar or `Ctrl+Shift+S` when cloud sync is enabled (see Chapter 4).
- **BigQuery menu:** Sync, change center, open console (as implemented in `app/ui/main_window.py`).
- **Auto-sync:** When enabled in config, startup can trigger a daily sync based on `last_sync_date` and configured hour (UTC).

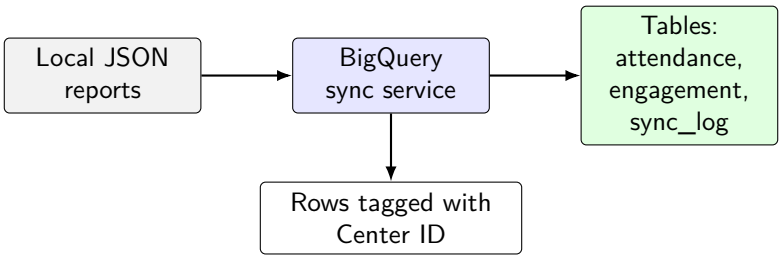

Figure 2.7: BigQuery sync: structured JSON is flattened and uploaded; every row carries Center ID for multi-site warehouses.

2.7 Keyboard shortcuts

Shortcut	Action
`Ctrl+O`	Open input folder
`Ctrl+R`	Open report
`Ctrl+,`	Settings
`Ctrl+1` / `2` / `3`	Classroom / Attendance / Report Viewer
`Ctrl+Shift+S`	Sync to BigQuery

2.8 Operational playbooks

2.8.1 New center onboarding

1. Install the app (portable build or installer).
2. Run once; set Center ID.
3. Confirm `bigquery.auto_sync` and `sync_hour` (UTC) in config match policy.
4. Train operators on watch-folder workflow and BUILD_DB vs. EVAL_DAY behavior.

2.8.2 Weak PCs and large files

- Increase **frame skip** in Settings.
- Disable **preview**.
- Prefer CPU-only PyTorch if GPU drivers are unstable.
- Reduce camera resolution or re-encode if memory errors persist.

Chapter 3

How it is made: technology stack

3.1 Platform and language

- **Runtime:** Python 3.9+.
- **Desktop GUI:** PyQt6 (`PyQt6`, `PyQt6-Qt6`).
- **Packaging:** PyInstaller (`build_exe.py`) produces an onedir-style layout under `dist\TataStriveAnalytics\` optional Inno Setup installer (`installer\`).

3.2 Computer vision and ML

Component	Role in product
PyTorch / torchvision	Tensor backend for YOLO and custom pipelines
Ultralytics YOLO	Person detection, pose, face weights
boxmot (BoTSORT-style)	Multi-object tracking / Re-ID path
InsightFace	Face analysis; models downloaded on first use
ONNX Runtime	Face embedding path; CPU/GPU wheels
OpenCV (`opencv-python`)	Decode video, preprocess, visualize
EasyOCR	Optional timestamp ROI reading (cross-day settings)
SciPy, NumPy	Numerics; NumPy version pinned for PyTorch compatibility (see `requirements_app.txt`).

3.3 Optional and cloud

- **Groq vision API** (`groq`): optional VLM metadata (e.g. classroom name, date/time) when enabled in the deployment.
- **Google BigQuery:** `google-cloud-bigquery`, `google-auth`; lazy import paths so the app can start without cloud libraries.
- **Intel OpenVINO:** optional acceleration (commented guidance in requirements).

3.4 Repository layout (maintainers)

A fuller directory-by-directory description (including `app/ui` files, `scripts/`, build outputs, and operator data paths) is in Chapter 4, Section 4.4.

Path	Purpose
app/	Application package (main.py, config, sync, updater)
app/ui/	Main window, tabs, dialogs, widgets
app/workers/	ClassroomWorker, CrossDayWorker (QThread)
app/bigquery_sync.py	Upload and schema mapping
app/updater.py	GitHub Releases polling and delta patch
app/config.py	ConfigManager, defaults, migrations
app/resources/	Styles, icons
classroom_analysis/	Supporting analysis / VLM helpers
unique_and_recognition.py	Script-style reference pipeline (batch/dev)
build_exe.py	PyInstaller build
installer/	Windows installer scripts
requirements_app.txt	Pinned dependency set for the desktop app

3.5 Architecture diagram

Figure 3.1 shows how UI, workers, pipelines, BigQuery, and updates relate.

3.6 Processing workers (conceptual)

- **ClassroomWorker:** Runs in a QThread; signals for progress, log lines, preview frames, finished report path, errors. Implements or delegates to face/engagement analyzer with callbacks; config keys under classroom and inference.
- **CrossDayWorker:** Wraps cross-day analyzer with callbacks; thresholds (t_strict_merge, t_new_id, etc.), optional motion, OCR ROI, annotated video output, delete-after-process; updates master database artifact.

3.7 Frozen executable notes

app/main.py sets CUDA_VISIBLE_DEVICES="" early on frozen Windows builds to reduce DLL issues; merges partial app/ overlay next to the exe for delta updates; optional ONNX preload ordering for DLL search paths.

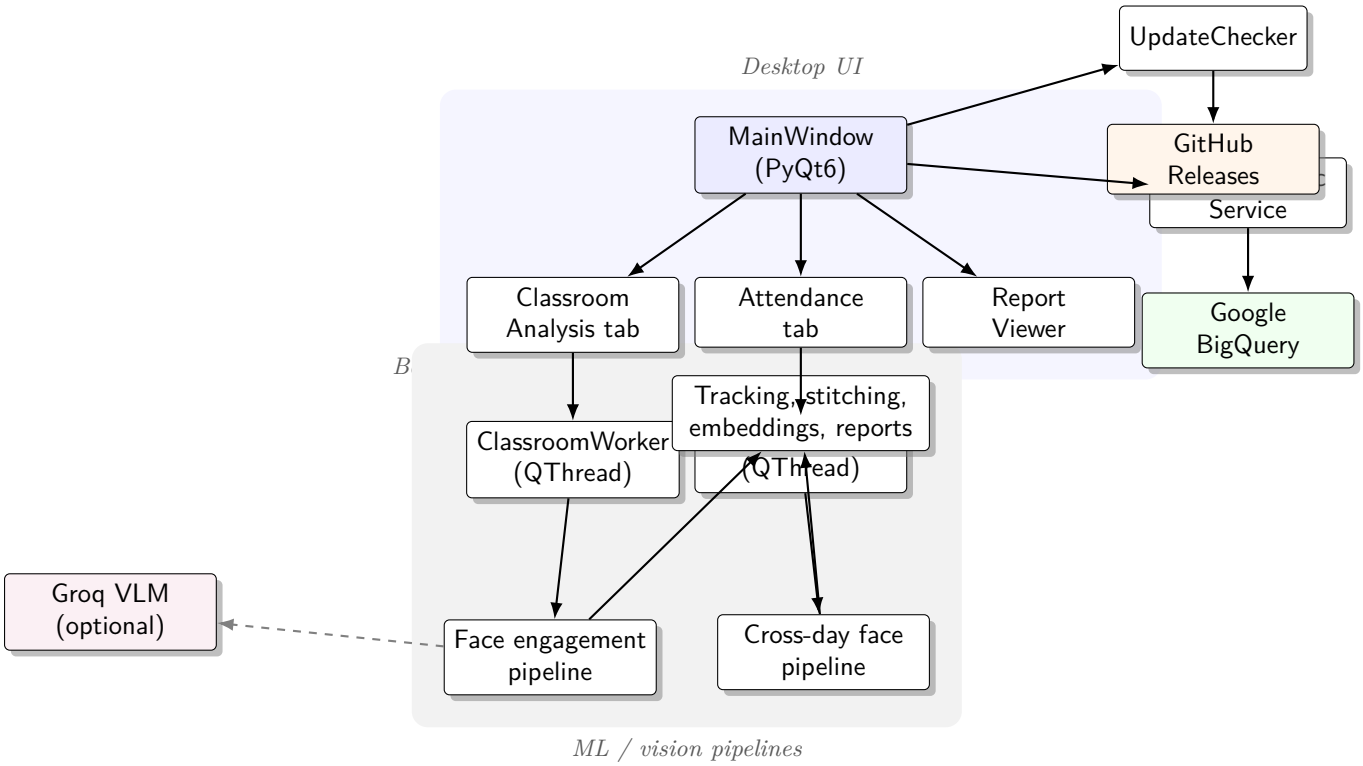

Figure 3.1: High-level architecture: PyQt6 UI drives background workers; workers run vision pipelines and write JSON artifacts; optional BigQuery sync and silent updates run alongside.

Chapter 4

Technical documentation

4.1 Installation from source

4.1.1 Prerequisites

- **Python** 3.9 or newer (64-bit recommended).
- **Git** (optional) for cloning the repository.
- **Visual C++ Redistributable** on Windows if PyTorch or ONNX DLLs fail to load.
- **GPU** optional; CPU-only PyTorch is supported and often preferred for classroom PCs.

4.1.2 Create environment and install dependencies

From the repository root:

```
python -m venv .venv
.venv\Scripts\activate
pip install --upgrade pip
pip install -r requirements_app.txt
```

On Linux/macOS, activate with `source .venv/bin/activate`.

4.1.3 Model files (`Models/`)

Place or download weights under `Models/`:

- `yolov8m.pt` — person detection (Ultralytics can fetch).
- `yolov8n-pose.pt` — pose (Ultralytics can fetch).
- `yolov8n-face.pt` — face detection; run `python download_face_model.py` from project root if missing.
- `osnet_x1_0_msmt17.pt` — Re-ID for BoTSORT; boxmot typically downloads on first use.
- InsightFace bundles download automatically on first cross-day use when the stack is healthy.

4.1.4 Launch (development)

```
python run_app.py
```

or

```
python -m app.main
```

4.2 Building a standalone executable

From the repository root:

```
python build_exe.py
```

Output layout: `dist/TataStriveAnalytics/` (PyInstaller onedir-style). Distribute the entire folder or wrap with the Windows installer (next section).

4.3 Windows installer (Inno Setup)

1. Build the compiled app: `python build_exe.py`.
2. Install Inno Setup 6 on the build machine (https://jrsoftware.org/isdl.php).
3. Run:

```
powershell -ExecutionPolicy Bypass -File .\installer\build_installer.ps1 -Version
    1.0.0
```

4. Share `release/TataStriveAnalytics_Setup_<version>.exe` with end users; they do not need Python.

4.4 Repository and folder structure

This section describes how the codebase and runtime directories are organized. Paths are relative to the **repository root** unless stated otherwise.

4.4.1 Top-level layout

The project is a single Python application plus supporting modules, build tooling, and documentation. A conceptual tree:

```
TataStriveFinal/
  app/                      # Main desktop application package
  classroom_analysis/       # Classroom pipeline helpers (VLM, stitching, etc.)
  scripts/                  # Maintenance: BigQuery, ETL, release utilities
  installer/                # Inno Setup, SQL patches, installer PowerShell
  docs/                     # LaTeX manual and other docs
  Screenshots/              # UI screenshots for documentation
  Models/                   # YOLO / Re-ID weights (often created at setup)
  .github/workflows/        # CI (e.g. auto_release)
  run_app.py                # Dev entry: launches the GUI
  build_exe.py              # PyInstaller build driver
  requirements_app.txt      # Pip dependencies for the app
  TataStriveAnalytics.spec # PyInstaller spec (referenced by build)
  download_face_model.py    # Fetches yolov8n-face weights
  unique_and_recognition.py # Standalone attendance-style script (reference)
```

Other files may appear at the root during development (sample JSON, logs, local databases); only stable, versioned paths are listed here.

4.4.2 Application package (`app/`)

The GUI and orchestration live under `app/`. Import root is the package name `app`.

Path	Role
app/__init__.py	Package metadata; __version__ (e.g. 3.0.0).
app/main.py	Process entry: QApplication, DPI, stylesheet, frozen-exe bootstrap, optional ONNX path setup, then main window.
app/config.py	ConfigManager: load/save JSON, defaults, migrations, dot-key access.
app/bigquery_sync.py	Serialize local reports and push to BigQuery tables.
app/updater.py	Poll releases, download delta ZIP, apply overlay, restart.
app/resources/	Static assets (e.g. styles.qss for Qt styling).

Table 4.1: Core app/ modules

User interface (app/ui/)

Module	Role
main_window.py	Main window: tabs, menus, toolbar, status bar, shortcuts, sync hooks.
classroom_tab.py	Classroom Analysis: folders, monitoring, preview, progress.
crossday_tab.py	Attendance: folders, database path, listener, preview.
report_viewer.py	Load JSON reports; table/tree; CSV export.
settings_dialog.py	Multi-tab settings bound to config.json sections.
center_dialog.py	First-run / change Center ID.
update_dialog.py	Update UX when not fully silent.
widgets/progress_panel.py	Progress bar and log area used by tabs.
widgets/video_preview.py	Optional frame preview widget.
widgets/file_picker.py	Reusable folder/file browse helpers.

Table 4.2: app/ui/ modules

Background workers (app/workers/)

- classroom_worker.py — QThread worker for classroom/engagement pipeline; emits progress, logs, preview frames, completion paths.
- crossday_worker.py — QThread worker for cross-day attendance; same signal pattern; updates master DB and writes attendance JSON (and optional video).

Both workers stay off the UI thread so the interface remains responsive during long video jobs.

4.4.3 Classroom analysis helpers (classroom_analysis/)

Python modules used by or aligned with the classroom engagement pipeline (probes, stitching, optional VLM metadata):

- stitch_logic.py, group_by_session.py — grouping and stitch-related logic.
- vlm_metadata.py — optional vision-language metadata extraction.
- requirements_classroom_activity.txt — optional extra deps for that sub-pipeline (if used standalone).

The desktop app primarily drives processing through workers that call into the integrated analyzer stack; these files support shared algorithms and experimentation.

4.4.4 Maintenance and ops scripts (`scripts/`)

Batch-oriented tools (run from repo root with appropriate Python env):

- BigQuery: `sync\_attendance\_json.py`, `clear\_bigquery.py`, `repair\_bigquery\_dedupe.py`, `bq\_attendance\_summary.py`, `clean\_bq\_attendance\_export.py`, `sync\_log\_insert\_queue.py`.
- Data: `etl\_student\_embeddings.py` (gallery / embedding prep).
- Release: `publish\_update.py` (maintainer publishing to GitHub Releases).

These are not required for day-to-day GUI operation but are part of the same product repository.

4.4.5 Installer and database SQL (`installer/`)

- `build\_installer.ps1`, `TataStriveAnalytics.iss` — build a Windows setup EXE from `dist/` output.
- `create\_release\_zip.ps1`, `install\_tatastrive.ps1` — auxiliary packaging or install flows.
- `bq\_add\_missing\_columns.sql`, `bq\_add\_sync\_log\_videos\_in\_queue.sql` — schema or migration helpers for the warehouse.
- `README.md` — installer prerequisites (Inno Setup) and commands.

4.4.6 Models directory (`Models/`)

At runtime the app expects heavyweight neural weights under `Models/` (repository root or resolved relative to cwd / frozen bundle). Typical files:

- `yolov8m.pt`, `yolov8n-pose.pt` — Ultralytics; may auto-download.
- `yolov8n-face.pt` — from `download_face_model.py` if not present.
- `osnet_x1_0_msmt17.pt` — Re-ID; often pulled by boxmot on first use.
- InsightFace model caches — usually under user cache or vendor default paths after first run.

The directory is often **gitignored** or empty in a fresh clone; populate it during environment setup.

4.4.7 Build outputs

- **PyInstaller onedir:** `dist/TataStriveAnalytics/` contains the executable, `_internal/` with bundled Python and DLLs, and resources. Distribute the *whole* folder or ship via the Inno installer.
- **Installer artifact:** `release/TataStriveAnalytics_Setup_<version>.exe` (path may vary with script version string).
- **Intermediate:** `build/` holds PyInstaller work files; safe to delete to reclaim disk space.

4.4.8 Per-user configuration (outside the repo)

Not stored in the repository; created at runtime on each machine:

- `~/.tatastrive/config.json` — persisted UI settings, last folders, completed-video lists, BigQuery schedule flags, inference defaults, window geometry.

Windows equivalent: user profile folder `.tatastrive\config.json`. This file is the single source of truth for operator preferences across sessions.

4.4.9 Operator-chosen directories (data plane)

The application does not mandate a fixed disk layout for video and results. Operators configure:

Concept	Description
Video input folder	Watch location for new `.mp4` / `.avi` / `.mkv` / `.mov` files; listener polls this tree.
Output directory	JSON reports, optional annotated MP4, crops, verification exports, updated master DB paths as configured.
Master DB path	Attendance: `.db` or `.pkl` gallery; may live on a share for multi-PC operations (organizational policy).

Table 4.3: Logical folders (paths stored in config and UI)

Idempotency: For each watch folder, the app records completed file paths in `config.json` so the same video is not processed twice after restart.

4.4.10 Typical output artifacts (by mode)

Written under the configured **output directory** (exact names depend on pipeline version and settings):

- **Classroom:** `class_dynamics_report.json`, `stitching_index.json`.
- **Attendance:** date-stamped `*_attendance_report.json`, optional `*_output.mp4`, directories such as `crops_*`, `Verification_Matches/`, updated `master_database` files (`.pkl` / `.db`).

Report Viewer and BigQuery ingestion rely on these filenames and JSON shapes staying compatible across releases.

4.4.11 Documentation assets

- `README_APP.md`, `PRODUCT_DOCUMENTATION.md`, `COMPREHENSIVE_CODE_DOCUMENTATION.md` — Markdown references in the repo root.
- `docs/TataStrive_LaTeX_Manual/` — this PDF project (`main.tex`, chapters, TikZ figures).
- `Screenshots/` — PNG captures referenced by the LaTeX manual via `\graphicspath`.

4.4.12 CI

`.github/workflows/auto_release.yml` (and any sibling workflows) automate tagging, builds, or release uploads; adjust branch and secret names to match your fork.

4.5 Configuration file

4.5.1 Location

The `ConfigManager` (`app/config.py`) persists settings to:

`~/.tatastrive/config.json` (Windows: user profile `.tatastrive\config.json`)

4.5.2 Top-level keys (summary)

Key	Meaning
`center_id`	Site tag for UI and BigQuery rows
`last_*`	Last-used paths for convenience
`classroom_completed_videos` / `crossday_completed_videos`	Per-folder finished lists (idempotent monitoring)
`bigquery`	`auto_sync`, `sync_hour` (UTC), `last_sync_date`
`classroom`	Probe timing, frame skip, stitching, optional delete video
`crossday`	Face thresholds, OCR ROI, motion, outputs, student DB path
`inference`	Device/backend, YOLO and face input sizes, frame skip, preview
`preview_enabled`	Global preview coordination
`window`	Geometry persistence

Settings UI groups mirror these sections (`app/ui/settings\_dialog.py`). The config layer supports dot-notation `get`/`set` and merges defaults when the file is upgraded; face default migrations use `face_match_defaults_revision`.

4.6 BigQuery integration

4.6.1 Module and dataset

Implementation: `app/bigquery\_sync.py`. As shipped in code, the project and dataset identifiers are embedded (see source for `PROJECT_ID`, `DATASET_ID`); rows always include `center_id` and sync timestamps.

4.6.2 Tables

- `attendance_reports` — flattened attendance JSON.
- `engagement_reports` — classroom engagement summaries.
- `sync_log` — per-file audit, errors, queue hints.

4.6.3 When sync runs

- **Manual:** toolbar / `Ctrl+Shift+S`.
- **Automatic:** if `bigquery.auto_sync` is true, startup enforces at-most-once per calendar day semantics using `last_sync_date` and scans recent output directories.

If required libraries are missing or sync is not enabled, upload is skipped gracefully and the UI remains usable.

4.7 Updates and releases

Figure 4.1 illustrates the silent update pipeline.

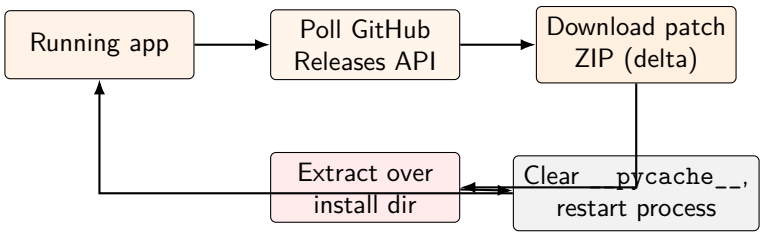

Figure 4.1: Silent update path: background check, delta patch application, cache cleanup, and process restart (see `app/updater.py`).

- **Module:** `app/updater.py`.
- **Source of truth:** GitHub Releases API for the configured repository (override with `TATASTRIVE_GITHUB_REPO` if supported in code).
- **Mechanism:** Download a patch ZIP (delta), extract over the install directory, clear relevant `__pycache__`, restart the process.
- **Operations:** Tune poll interval for production to limit API traffic; publish releases and patch assets per internal runbook (`scripts/publish\_update.py` may exist for maintainers).

4.8 Security and privacy

- Raw video and face crops remain on operator-chosen disk paths; BigQuery receives structured fields from JSON schemas, not full video uploads.
- Center ID is a tenant tag, not a cryptographic control — combine with IAM and dataset policies in GCP.

4.9 Troubleshooting

Symptom	Remediation
Analysis tabs disabled	PyTorch DLL failure; install CPU PyTorch or VC++ redist; see `README\_APP.md`.
NumPy errors	Pin `numpy<2` with older PyTorch, or upgrade PyTorch for NumPy 2.x.
ONNX / InsightFace DLL	`pip install onnxruntime` (CPU) or run `fix_onnxruntime.bat`; simplified track-only mode may apply.
BigQuery no rows	Confirm cloud sync is enabled, dependencies are installed, and deployment policy allows upload.
Same video re-queued	Watch folder path must match persisted completed-video lists in config.
CUDA OOM	Increase frame skip, disable preview, reduce resolution.

4.10 How to update the application or dependencies

4.10.1 Developer workstation

```
git pull
pip install -r requirements_app.txt --upgrade
python run_app.py
```

Resolve merge conflicts in `requirements_app.txt` carefully; validate PyTorch/NumPy pairing.

4.10.2 Frozen deployment

1. Build new artifacts with `python build_exe.py`.
2. Re-run Inno Setup script with an incremented version string.
3. Alternatively rely on in-app delta updates if your release pipeline publishes compatible patch ZIPs to GitHub Releases.

4.10.3 Configuration migration

Back up `config.json` before major upgrades. The app merges new defaults; verify `center_id`, paths, and BigQuery flags after upgrade.

4.11 Related documentation in the repository

- `README_APP.md` — quick install, shortcuts, troubleshooting.
- `PRODUCT_DOCUMENTATION.md` — product and architecture narrative.
- `installer/README.md` — installer build steps.
- `COMPREHENSIVE_CODE_DOCUMENTATION.md` — extended code-level notes (if present).

Appendix A

Version and license

TataStrive Analytics version 3.0.0 is defined in `app/__init__.py`. License: proprietary — TataStrive (see `README_APP.md`).